

Réseaux bancaires et contagion

Simuler une crise, prédire son ampleur, évaluer une intervention

Dr. Mario Veruete

Coruscant / Willing Technologies

Hackathon Data Science & Risque, Université de Bordeaux

Cours de lancement, 25 septembre 2026

Version étudiants

Partie	Durée	À la fin, vous saurez...
0 Ouverture		situer la mission, les rendus et le kit de départ
1 Contexte historique et économique	10 min	expliquer comment une crise se transmet
2 Graphes orientés et expositions	25 min	lire une arête, calculer degrés et poids, contrôler
3 Pertes, capital, cascade à seuil	40 min	appliquer la règle à la main, éviter les pièges
4 Machine learning	30 min	table des scénarios, protocole, MAE
5 Interventions et exemple RELAIS	15 min	comparer des stratégies à budget identique
6 Organisation		connaître les jalons, les rendus et la grille
A Annexes		glossaire, fiche mémo, bibliographie

Jalon de fin de séance (sujet)

Chaque équipe exécute le chargement des données, explique le sens des liens et dispose d'une première figure.

- Ce support sert pendant le cours et après : il se lit seul, sans l'oral.
- Chaque terme technique est **défini** (en gras bleu) à sa première apparition, puis repris dans le **glossaire** (annexe).
- Formules, conventions et contrat des fonctions : voir la **fiche mémo** (annexe).
- Virgule décimale dans le texte (0,3), point dans le code (0.3).
- **Version étudiants** : le simulateur est décrit en pseudo-code ; votre équipe l'écrit en Python.

Code couleur des formules (cours, figures, démonstrations)

λ	fraction de perte (orange)
e_{ij}	montant que i doit à j (bleu)
c_j	capital de la banque j (vert)
D_t	banques en défaut à l'étape t (rouge)
L_j	perte subie par j (violet)
$y, \hat{y}$	cible et prédiction (sarcelle)
v_j, s_j, B	fragilité, score, budget (brun)

Dans le code affiché, les variables **lam**, **defauts** et **perte** reprennent ces couleurs.

Des banques se prêtent de l'argent. Si l'une ne rembourse plus, ses prêteurs perdent ; si leurs pertes dépassent ce qu'ils peuvent absorber, ils font défaut à leur tour : c'est une **cascade de défauts**. Une cellule de supervision veut repérer les situations fragiles et comparer des façons de renforcer le système avec un budget limité.

La question du hackathon

À partir de la structure d'un réseau (un *graphe*) et des caractéristiques des banques, peut-on prévoir l'ampleur d'une cascade de défauts et proposer une intervention qui en réduit les conséquences ?

Quatre étapes (12 réseaux fictifs de 60 banques) :

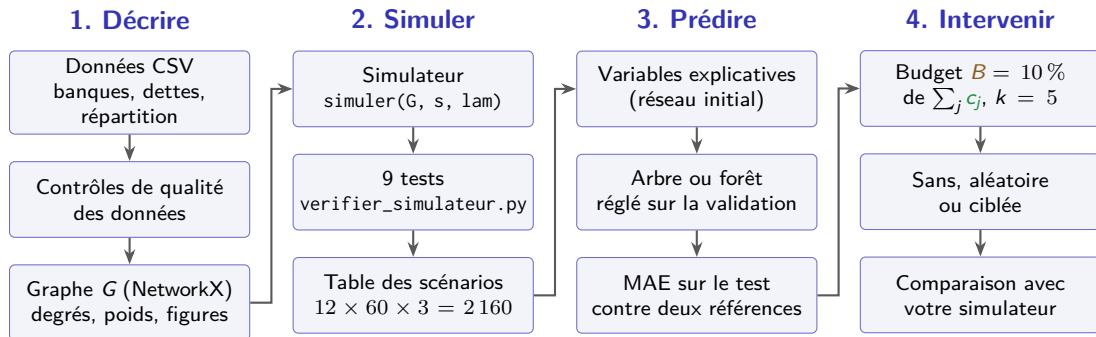
- ① **Décrire** le réseau, repérer des banques influentes ou fragiles (partie 2)
- ② **Simuler** la propagation d'un défaut avec une règle commune (partie 3)
- ③ **Prédire** la proportion de défauts secondaires par machine learning (partie 4)
- ④ **Comparer** une intervention ciblée à une intervention aléatoire, à budget égal (partie 5)

Résultat attendu : une étude *reproductible et argumentée*, appuyée sur quelques figures lisibles.

- **Dette** et **créance** : les deux faces d'un même prêt. Si i a emprunté à j , i a une dette envers j et j détient une créance sur i .
- **Débiteur** : celui qui doit (i). **Créancier** : celui à qui l'on doit (j).
- **Contrepartie** : l'autre partie d'un contrat financier (prêt, dérivé, assurance...).
- **Exposition** : ce qu'un créancier peut perdre si son débiteur ne paie pas ; ici e_{ij} .
- **Défaut** : la banque ne peut plus faire face à ses engagements. Ici : ses pertes dépassent son capital, ou elle subit le choc initial.
- **Capital** (fonds propres) : coussin qui absorbe les pertes ; noté c_j .
- **Contagion** : transmission d'une difficulté d'une banque à d'autres.
- **Choc initial** : défaut imposé d'une banque d'origine s ; il ouvre un **scénario**.

En une phrase

i doit e_{ij} à j ; si i fait défaut, j perd une partie de cette créance ; si la perte de j dépasse son capital c_j , j fait défaut à son tour, et ainsi de suite.



Chaque étape s'appuie sur la précédente : un simulateur *vérifié* est la condition de tout le reste, car la table des scénarios et la comparaison des interventions en dépendent.

Téléchargement : github.com/marioveruetephd/hackathon-contagion-bancaire (bouton **Code**, puis **Download ZIP**).

Fichier (dépôt GitHub du hackathon)	Contenu
README.md	mode d'emploi (page d'accueil du dépôt)
sujet/, cours/	le sujet du hackathon et ce cours (PDF)
mon_simulateur.py	le fichier que votre équipe complète (simulateur)
donnees/banques.csv	une ligne par banque : reseau_id, banque_id, capital
donnees/expositions.csv	une ligne par dette : debiteur_id, creancier_id, montant
donnees/repartition.csv	une ligne par réseau : reseau_id, usage (8 / 2 / 2 réseaux)
donnees/tirages_aleatoires.csv	les 10 tirages aléatoires officiels (graines 0 à 9)
donnees/dictionnaire_variables.*	la définition de chaque colonne (.md et .csv)
donnees/exemples/	les exemples A-B-C (3 banques) et RELAIS (12 banques)
donnees/brutes/	données avec anomalies volontaires, pour s'entraîner aux contrôles
aides/aides_contagion.py	les fonctions d'aide (diapositive suivante)
aides/verifier_simulateur.py	les 9 tests de votre simulateur

Montants en unité monétaire fictive. Les deux exemples sont exclus de l'étude. Aucune autre donnée n'est nécessaire.

Aide (aides_contagion.py)	Rôle
<code>depasse_capital(perte, capital)</code>	test « $\text{perte} > \text{capital}$ » robuste aux arrondis
<code>proportion_secondaire(defaults, n)</code>	la mesure y (partie 3)
<code>generer_scenarios(...)</code>	les 2160 scénarios, calculés avec vos fonctions
<code>references_medianes, predire_references</code>	les deux références du protocole (partie 4)
<code>capitaux_renforces(G, choisies)</code>	copie de G renforcée avec le budget (partie 5)
<code>charger_tirages, tirer_banques_aleatoires</code>	les tirages aléatoires reproductibles
<code>comparer_interventions, resumer_interventions</code>	la comparaison des stratégies

À écrire par votre équipe

`construire_graphe(banques_r, expositions_r)` et `simuler(G, origine, lam)`. Les aides reçoivent vos fonctions en argument. Import depuis un notebook : `import sys; sys.path.append("aides"), puis import aides_contagion as ac (chemin à adapter).`

- 0 Ouverture
- 1 Contexte historique et économique**
- 2 Graphes orientés et expositions
- 3 Pertes, capital, cascade à seuil
- 4 Machine learning
- 5 Interventions et exemple RELAIS
- 6 Organisation

1998 : LTCM, un fonds trop connecté pour tomber seul

- **Long-Term Capital Management** : **fonds spéculatif** (*hedge fund*) américain créé en 1994 ; deux lauréats du prix Nobel d'économie 1997 parmi ses associés.
- Arbitrages très endettés : un **levier** (actifs rapportés aux fonds propres) de l'ordre de 25 début 1998, et de gros contrats dérivés avec les grandes banques.
- Août 1998 : défaut de la Russie, fuite vers les titres sûrs ; LTCM perd l'essentiel de ses fonds propres en quelques semaines.
- **23 septembre 1998** : à l'initiative de la **Fed de New York**, un consortium de 14 banques et maisons de titres apporte **environ 3,6 Md\$** et reprend 90 % du fonds. Pas d'argent public.
- Motif : éviter une liquidation désordonnée, qui aurait infligé des **pertes directes** aux contreparties et forcé des ventes massives d'actifs.

Leçon

Ce qui compte n'est pas seulement la solidité de chaque établissement, mais le **réseau des expositions** qui les relie.

15 septembre : faillite de Lehman Brothers

- Quatrième banque d'investissement américaine, plus grande faillite de l'histoire des États-Unis (plus de 600 Md\$ d'actifs). Pas de sauvetage public.
- Pertes directes pour ses créanciers. Exemple : le fonds monétaire *Reserve Primary Fund*, qui détenait des titres de Lehman, voit sa part passer sous 1 \$ (16 septembre).

16 septembre : sauvetage d'AIG

- La Fed prête jusqu'à **85 Md\$** à l'assureur AIG, contre environ 80 % de son capital ; le soutien public atteindra environ 180 Md\$ d'engagements.
- AIG avait vendu à de grandes banques des **CDS** (*credit default swaps*), assurances contre le défaut d'un emprunteur : sa faillite leur aurait causé des pertes directes.

Exposition de contrepartie

Risque de contrepartie : risque que l'autre partie d'un contrat ne paie pas ce qu'elle doit. Lehman : un défaut qui se propage ; AIG : un sauvetage pour éviter qu'il se propage.

- **Marché interbancaire** : les banques s'y prêtent entre elles, souvent au jour le jour, pour ajuster leur trésorerie.
- Premières tensions le 9 août 2007 : BNP Paribas gèle trois fonds exposés aux crédits hypothécaires américains ; la BCE injecte près de 95 Md€ en une journée.
- Après Lehman, chaque banque se demande *qui est exposé à qui*. Faute de le savoir, elle garde ses liquidités et cesse de prêter sans garantie.
- Indicateur : l'**écart LIBOR-OIS** à 3 mois, prime exigée pour prêter à une autre banque sans garantie. En dollars, proche de 0,1 point avant août 2007, il dépasse 3,5 points en octobre 2008.
- Réponses : liquidités des banques centrales, baisse coordonnée des taux (8 octobre 2008), garanties publiques des prêts interbancaires, recapitalisations.

Leçon pour le hackathon

L'opacité du réseau nourrit la peur de la contagion. Savoir **qui doit combien à qui** est un enjeu de supervision (Haldane, 2009) : c'est l'objet que nous allons décrire et simuler.

Canal direct, par les bilans

i doit e_{ij} à j . Si i fait défaut, j perd une partie de sa créance ; si cette perte dépasse son capital c_j , j tombe à son tour, et touche ses propres créanciers.

Seul canal du simulateur : les **expositions directes** (prêts entre banques, dérivés, dépôts).

Canaux indirects : non modélisés

- **Ventes forcées** (*fire sales*) : une banque en difficulté vend en urgence ; les prix baissent ; les détenteurs des mêmes actifs perdent.
- **Expositions communes** : des banques sans lien direct détiennent les mêmes actifs et subissent le même choc.
- **Liquidité** : les prêteurs retirent leurs financements ; une banque solvable peut manquer de liquidités.
- **Panique et information** : un défaut fait douter de banques semblables.

Conséquence

Le simulateur isole *un seul* mécanisme : il sous-estime la contagion réelle.

Ce que dit la littérature (1/2) : les fondements

- **Allen et Gale (2000)** : la *structure* des créances compte. Un réseau complet (chaque banque prête à toutes) répartit mieux les pertes qu'un anneau (chacune prête à sa voisine).
- **Eisenberg et Noe (2001)** : les paiements de toutes les banques se déterminent ensemble, comme un *point fixe*; une banque en difficulté paie une partie de ses dettes. Notre règle est plus simple : fraction de perte fixe λ , pas de paiements partiels.
- **Furfine (2003)** : expositions observées sur le marché interbancaire américain au jour le jour ; en simulant des défauts, le risque de contagion apparaît faible, sauf avec des taux de perte élevés.
- **Upper et Worms (2004)** : marché allemand ; faute de données bilatérales, la matrice des expositions est reconstituée à partir des totaux de bilan. La contagion est possible, d'autant plus étendue que le taux de perte est élevé.

Ce que nous en retenons

L'ampleur des pertes transmises (notre λ) et la forme du réseau sont les deux ingrédients de la contagion directe.

- **Gai et Kapadia (2010)** : cascades à *seuil* sur des réseaux aléatoires, avec une règle très proche de la nôtre. Résultat « **robuste mais fragile** » : la contagion est rare, mais quand elle part, elle peut toucher une grande partie du système.
- **Haldane (2009)**, Banque d'Angleterre : le système financier comme réseau complexe ; il faut cartographier les expositions et surveiller les établissements les plus connectés.
- **Battiston et al. (2012)**, *DebtRank* : impact systémique d'une banque, en propageant la détresse et pas seulement les défauts (« trop central pour faire faillite »).
- **Craig et von Peter (2014)** : le marché interbancaire allemand a une structure **cœur-périphérie** : un petit cœur très interconnecté, une large périphérie reliée surtout au cœur.
- **Acemoglu, Ozdaglar et Tahbaz-Salehi (2015)** : des liens denses amortissent les petits chocs mais propagent les grands.

Rappel

Notre simulateur est un **mécanisme pédagogique simplifié**, inspiré de ces travaux : il ne prévoit rien sur des banques réelles.

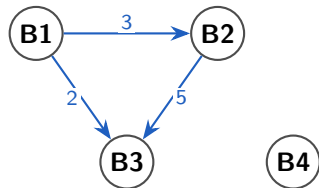
- 0 Ouverture
- 1 Contexte historique et économique
- 2 Graphes orientés et expositions**
- 3 Pertes, capital, cascade à seuil
- 4 Machine learning
- 5 Interventions et exemple RELAIS
- 6 Organisation

Un graphe orienté pondéré

Pour étudier la contagion, il faut une carte : *qui doit combien à qui* ? La théorie des graphes en fournit le vocabulaire.

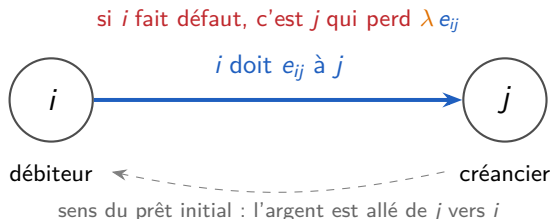
- **Graphe** : des **sommets** (ici les banques) reliés par des **arêtes** (ici les dettes).
- **Orienté** : chaque arête a un sens, une flèche $i \rightarrow j$; les arêtes $i \rightarrow j$ et $j \rightarrow i$ sont distinctes.
- **Pondéré** : chaque arête porte un nombre, son **poids**; ici le montant de la dette $e_{ij} > 0$.
- Si l'arête $i \rightarrow j$ existe, j est un **successeur** de i et i un **prédécesseur** de j . Un **chemin** est une suite d'arêtes qui s'enchaînent ($i \rightarrow j \rightarrow k$).

Dans le jeu de données : un graphe par réseau, $N = 60$ sommets, une arête par dette.



B1 doit 3 à B2 et 2 à B3; B2 doit 5 à B3.
B2 est un successeur de B1; $B1 \rightarrow B2 \rightarrow B3$ est un chemin.
B4 n'a aucun lien : c'est une banque isolée.

La convention du sujet : la perte suit la flèche

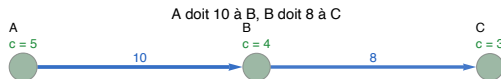


- **Convention** : l'arête $i \rightarrow j$ de poids e_{ij} signifie que i **doit** le montant e_{ij} à j .
- i est le débiteur, j le créancier : j détient une créance de e_{ij} sur i , c'est son exposition envers i .
- Si i fait défaut, c'est j qui risque de perdre : la flèche suit le **sens de propagation des pertes**.

Piège classique

Dessiner la flèche dans le sens du prêt ($j \rightarrow i$). Dans le sujet et dans le code, la flèche va du **débiteur** vers le **créancier** : `G.add_edge(debiteur_id, creancier_id, montant=...)`.

L'exemple A-B-C et la matrice d'exposition



A doit 10 à B ; B doit 8 à C ; $c_A = 5$, $c_B = 4$, $c_C = 3$ (fichiers exemples/abc_*.csv).

La **matrice d'exposition** E est un tableau $N \times N$: ligne i = débiteur, colonne j = créancier, case $(i, j) = e_{ij}$ (0 sans dette). Somme de la ligne i : dettes de i ; de la colonne j : créances de j . Diagonale nulle (pas de dette envers soi-même). Presque toutes les cases sont nulles : la matrice est **creuse**.

débiteur ↓ créancier →	A	B	C	dettes
A	0	10	0	10
B	0	0	8	8
C	0	0	0	0
créances	0	10	8	

B est à la fois créancier (de A) et débiteur (de C) : c'est un *intermédiaire*, par lequel une perte peut passer.

Degrés et poids : combien de liens, combien d'argent

$d_{\text{in}}(j)$ = nombre de **débiteurs** de j (arêtes entrantes)

$w_{\text{in}}(j) = \sum_i e_{ij} =$ **créances** détenues par j

$d_{\text{out}}(j)$ = nombre de **créanciers** de j (arêtes sortantes)

$w_{\text{out}}(j) = \sum_k e_{jk} =$ **dettes** de j

où e_{ij} : montant que i doit à j (0 s'il n'y a pas de dette); $\sum_i$ parcourt les débiteurs de j , $\sum_k$ ses créanciers
 d, w : *degré* = nombre d'arêtes; *poids* = somme des montants; « in » = entrant, « out » = sortant

Indicateur	Rôle dans la contagion	A	B	C
$d_{\text{in}}(j)$	nombre de défauts qui peuvent toucher j directement	0	1	1
$w_{\text{in}}(j)$	perte maximale de j si tous ses débiteurs tombent : $\lambda w_{\text{in}}(j)$	0	10	8
$d_{\text{out}}(j)$	nombre de banques touchées directement si j tombe	1	1	0
$w_{\text{out}}(j)$	pertes que le défaut de j inflige au total : $\lambda w_{\text{out}}(j)$	10	8	0

- **Banque isolée** : aucune dette et aucune créance ($d_{\text{in}}(j) = d_{\text{out}}(j) = 0$). Chaque réseau en contient quelques-unes.
- Elle ne transmet ni ne subit aucune perte : si elle est l'origine, la cascade s'arrête aussitôt ; sinon, elle ne fait jamais défaut.
- Elle compte pourtant dans N : la mesure $y = (|D_{\text{final}}| - 1)/(N - 1)$ de la partie 3 divisé par $N - 1 = 59$. Et elle sert d'origine comme les autres.

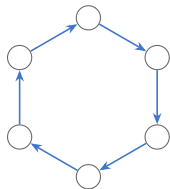
Piège : construire le graphe à partir des seules dettes

Un graphe fabriqué à partir des seules arêtes (par exemple `nx.from_pandas_edgelist`) **perd les banques isolées** : N trop petit, y surestimé, scénarios manquants. Avec 3 banques oubliées, 10 défauts secondaires donneraient $10/56 \approx 0,179$ au lieu de $10/59 \approx 0,169$.

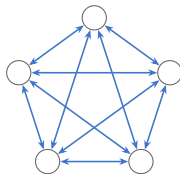
Bonne pratique

Ajouter d'abord **tous les sommets** à partir de `banques.csv` (`G.add_node`), puis les arêtes à partir de `expositions.csv`.

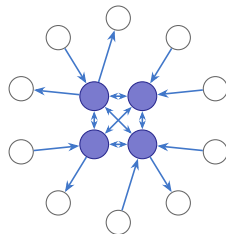
Trois structures typiques de réseaux



Anneau



Complet



Cœur-périphérie

- **Anneau** : chacun doit à son voisin ; les pertes passent de proche en proche (Allen et Gale).
- **Complet** : chacun doit à tous ; une perte est partagée entre de nombreux créanciers.
- **Cœur-périphérie** : un petit cœur de banques très liées entre elles, une périphérie reliée surtout au cœur. C'est la structure la plus souvent observée sur les marchés interbancaires (Craig et von Peter, 2014).

Les 12 réseaux du jeu de données ont des structures et des niveaux de fragilité *variés* : les décrire fait partie de votre travail (étape 1).

Contrôler les données avant tout calcul

Contrôle	Outil pandas (b = banques, e = expositions ; résultat attendu : 0)
Unicité de chaque couple (réseau, banque)	<code>b.duplicated(["reseau_id", "banque_id"]).sum()</code>
Aucune valeur manquante	<code>b.isna().sum(), e.isna().sum()</code>
Positivité : capital > 0, montant > 0	<code>(b["capital"] <= 0).sum(), (e["montant"] <= 0).sum()</code>
Débiteur et créancier existent dans le même réseau	<code>e.merge(b, ..., how="left", indicator=True)</code> puis compter les lignes « left_only »
Pas de dette envers soi-même	<code>(e["debiteur_id"] == e["creancier_id"]).sum()</code>
Pas de doublon d'une même paire orientée	<code>e.duplicated(["reseau_id", "debiteur_id", "creancier_id"]).sum()</code>
Répartition : 12 réseaux, 8 / 2 / 2	<code>repartition["usage"].value_counts()</code>

Données principales et données brutes

Le jeu principal (donnees/) doit passer tous ces contrôles : vérifiez-le et dites-le. Les fichiers de donnees/brutes/ contiennent des anomalies volontaires : entraînez-vous à les détecter, à les compter et à décider quoi en faire (corriger, écarter, signaler).

Python : charger les données et choisir un réseau

```
import pandas as pd
import networkx as nx
```

```
banques = pd.read_csv("donnees/banques.csv")
expositions = pd.read_csv("donnees/expositions.csv")
repartition = pd.read_csv("donnees/repartition.csv")
```

```
# Un réseau d'apprentissage pour explorer (le premier de la liste)
r = repartition.loc[repartition["usage"] == "apprentissage", "reseau_id"].iloc[0]
banques_r = banques[banques["reseau_id"] == r]
expositions_r = expositions[expositions["reseau_id"] == r]
print(r, len(banques_r), "banques,", len(expositions_r), "dettes")
```

- `pd.read_csv` lit un fichier CSV dans un **DataFrame** : un tableau à lignes et colonnes nommées.
- `df[df["reseau_id"] == r]` garde les lignes du réseau `r`; `.loc[condition, colonne]` choisit lignes et colonne. Chemins écrits depuis la racine du kit.

Python : construire le graphe d'un réseau

```
def construire_graphe(banques_r, expositions_r):
    """Graphe orienté pondéré d'un réseau : arête i -> j = « i doit montant à j »."""
    G = nx.DiGraph()
    for b in banques_r.itertuples(index=False):
        G.add_node(b.banque_id, capital=float(b.capital)) # garde aussi les banques isolées
    for e in expositions_r.itertuples(index=False):
        G.add_edge(e.debitteur_id, e.creancier_id, montant=float(e.montant))
    return G

G = construire_graphe(banques_r, expositions_r)
print(G.number_of_nodes(), G.number_of_edges()) # 60 banques, nombre de dettes
```

- 1 `nx.DiGraph()` crée un graphe **orienté** vide (`nx.Graph()` ignorerait le sens).
- 2 Un sommet par banque, avec l'attribut `capital`, **y compris les banques isolées**.
- 3 Une arête **débiteur** → **créancier** par dette, avec l'attribut `montant`.
- 4 `itertuples` parcourt les lignes; `b.banque_id` lit une colonne de la ligne `b`.

Contrat commun : attributs "capital" et "montant"; les aides et les tests s'appuient dessus.

```
j = "B07"
G.in_degree(j)           # d_in(j) : nombre de débiteurs de j
G.out_degree(j)          # d_out(j) : nombre de créanciers de j
G.in_degree(j, weight="montant") # w_in(j) : créances détenues par j
G.out_degree(j, weight="montant") # w_out(j) : dettes de j
list(G.predecessors(j))   # débiteurs de j (arêtes i -> j)
list(G.successors(j))    # créanciers de j (arêtes j -> k)
G.nodes[j]["capital"]    # c_j ; et G[i][j]["montant"] = e_ij si l'arête existe

indicateurs = pd.DataFrame({
    "degre_entrant": dict(G.in_degree()),
    "degre_sortant": dict(G.out_degree()),
    "poids_entrant": dict(G.in_degree(weight="montant")),
    "poids_sortant": dict(G.out_degree(weight="montant")),
})
indicateurs.sort_values("poids_sortant", ascending=False).head()
```

Une ligne par banque : de quoi repérer les banques très connectées et faire une première figure (histogramme des degrés, dessin avec `nx.draw`).

Le sujet demande deux hypothèses, formulées à partir du graphe *avant* toute simulation. Exemples :

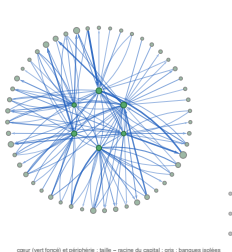
- ❶ « Une banque qui a de nombreux créanciers ($d_{\text{out}}(s)$ élevé) propage davantage de défauts. »
- ❷ « Un choc sur une banque dont les créanciers sont fragiles (beaucoup de créances pour peu de capital) produit une cascade plus grande. »
- ❸ « L'effet de la structure dépend de λ : peu de cascades à $\lambda = 0,3$, beaucoup plus à $\lambda = 0,9$. »

Une bonne hypothèse a trois ingrédients

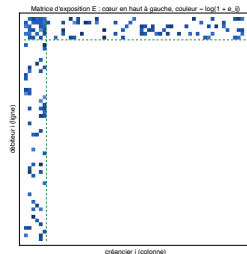
- un **mécanisme** économique : pourquoi cela devrait-il être vrai ?
- une **grandeur mesurable** sur le réseau initial : laquelle, calculée comment ?
- un **test** : quelle figure ou quel tableau, obtenu avec le simulateur, la confirmerait ou l'infirmerait ?

Une hypothèse infirmée par les simulations est un résultat à part entière : dites-le et cherchez pourquoi.

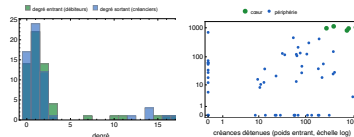
Démonstration en direct : notebook wolfram/Demos_Contagion.nb, **Démo 1 : exploration**



le réseau



la matrice d'exposition



les degrés et les poids

Ce qu'on va observer

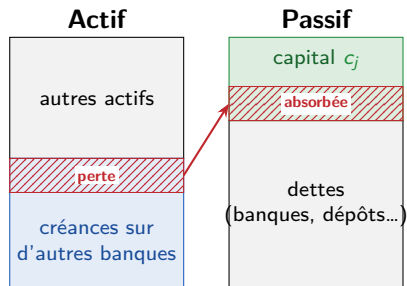
Un réseau de démonstration **hors base** (il ne dévoile rien du jeu de données) : dessin du graphe, matrice E (lignes débiteurs, colonnes créanciers), distribution des degrés et des poids.

Questions à se poser

Quelles banques ont le plus de créanciers ? Les gros créanciers sont-ils aussi de gros débiteurs ? Où sont les banques isolées ? Voit-on un groupe très inter-connecté ?

- 0 Ouverture
- 1 Contexte historique et économique
- 2 Graphes orientés et expositions
- 3 Pertes, capital, cascade à seuil**
- 4 Machine learning
- 5 Interventions et exemple RELAIS
- 6 Organisation

Le capital, un coussin contre les pertes



Une créance perdue (hachures) réduit l'actif ; les dettes restent dues ; le capital baisse d'autant.

- Le **capital** (fonds propres) est ce qui reste quand on retire des actifs toutes les dettes : $\text{capital} = \text{actifs} - \text{dettes}$.
- Une créance non remboursée fait baisser l'actif, mais la banque doit toujours ses propres dettes : c'est le capital qui **absorbe la perte**.
- $\text{Perte} \leq \text{capital}$: la banque reste **solvable** (ses actifs couvrent ses dettes). $\text{Perte} > \text{capital}$: elle devient **insolvable** ; dans notre modèle, elle **fait défaut**.
- Dans les données, c_j est directement le montant de pertes que j peut absorber : pas de bilan complet.

- Quand un débiteur fait défaut, son créancier récupère souvent une partie de sa créance (liquidation, garanties) : il ne perd pas tout.
- **Fraction de perte** $\lambda \in [0, 1]$: part de la créance perdue ; $1 - \lambda$ est le **taux de recouvrement**. En gestion des risques, on parle de perte en cas de défaut (*LGD, loss given default*).

perte de j sur sa créance envers i en défaut = $\lambda \times e_{ij}$

exemple : $\lambda = 0,5$ et $e_{AB} = 10$: B perd $0,5 \times 10 = 5$

- Dans un scénario, λ est **la même pour toutes les dettes**. Valeurs étudiées : $\lambda \in \{0,3 ; 0,6 ; 0,9\}$ (constante LAMBDA des aides).
- Cas extrêmes : $\lambda = 0$, aucune perte donc aucune contagion ; $\lambda = 1$, perte totale de la créance.
- Conséquence utile : si $\lambda w_{in}(j) \leq c_j$, la banque j ne peut **jamais** faire défaut à ce λ , même si tous ses débiteurs tombent.

La perte totale d'une banque : $L_j(D_t)$

Intuition : à une étape donnée, plusieurs débiteurs de j peuvent déjà être en défaut ; j perd sur chacun d'eux, et ces pertes s'additionnent.

$$L_j(D_t) = \lambda \sum_{i \in D_t} e_{ij}$$

où

- $L_j(D_t)$: perte **totale** de la banque j due à toutes les banques déjà en défaut
- λ : fraction de perte, la même pour toutes les dettes du scénario
- D_t : ensemble des banques en défaut à l'étape t ; au départ $D_0 = \{s\}$
- $\sum_{i \in D_t}$: somme sur les banques i de D_t ; seules comptent celles qui doivent quelque chose à j
- e_{ij} : montant que i doit à j ; une dette absente vaut zéro

- **Totale** : à chaque étape, on recalcule la somme sur tout D_t ; on ne l'ajoute pas à la perte de l'étape précédente.
- Exemple : j a trois débiteurs, dont deux en défaut qui lui doivent 4 et 6 ; avec $\lambda = 0,6$:
 $L_j = 0,6 \times (4 + 6) = 6$.

La règle de défaut : D_{t+1}

Intuition : une banque encore active fait défaut dès que sa perte totale dépasse son coussin ; les défauts déjà connus restent acquis.

$$D_{t+1} = D_t \cup \{j \notin D_t : L_j(D_t) > c_j\}$$

où D_t : banques déjà en défaut : elles le restent (défaut irréversible)

$\cup$: réunion : on ajoute les nouvelles banques à l'**ensemble** (collection sans ordre ni répétition)

$j \notin D_t$: banques encore actives : les seules que l'on teste

$L_j(D_t) > c_j$: la perte totale dépasse **strictement** le capital de j

D_{t+1} : défauts connus à l'étape suivante

En mots

On garde tous les défauts connus et on y ajoute, **en une seule fois**, toutes les banques actives dont la perte totale dépasse strictement le capital. On recommence tant que l'ensemble change.

Quatre principes, et pourquoi la cascade s'arrête

- ❶ **Défaut strict** : $L_j(D_t) = c_j$ ne déclenche pas de défaut ($0,5 \times 10 = 5 = c$: la banque tient, de justesse).
- ❷ **Choc initial imposé** : $D_0 = \{s\}$, quel que soit le capital de s .
- ❸ **Ajouts simultanés** : tous les nouveaux défauts d'une étape sont calculés à partir du même D_t , puis ajoutés ensemble.
- ❹ **Irréversibilité** : une banque en défaut le reste, donc $D_t \subseteq D_{t+1}$ (inclusion).

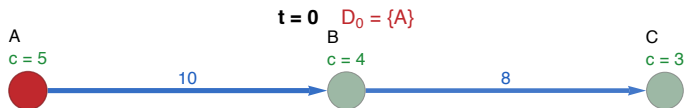
Comme les défauts s'accumulent, la perte $L_j(D_t)$ d'une banque active ne peut qu'augmenter d'une étape à l'autre : une banque qui tient à l'étape t peut tomber plus tard. Une **étape** désigne une application de la règle.

Arrêt : le point fixe

On s'arrête quand $D_{t+1} = D_t$. Un tel ensemble est un **point fixe** de la règle : l'appliquer encore ne change plus rien.

Chaque étape qui n'est pas la dernière ajoute au moins une banque, et il n'y a que $N - 1$ banques à ajouter : la cascade s'arrête en **au plus $N - 1$ étapes**.

Données : A doit 10 à B ; B doit 8 à C ; $c_A = 5$, $c_B = 4$, $c_C = 3$; $\lambda = 0,5$; origine $s = A$.



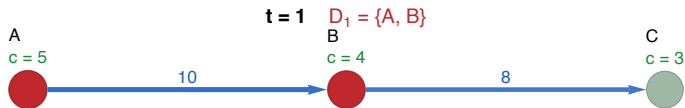
Étape 1 : on part de $D_0 = \{A\}$ (choc imposé) et on teste les banques actives B et C.

banque j	débiteurs de j dans D_0	perte $L_j(D_0)$	capital c_j	verdict
B	A, qui lui doit $e_{AB} = 10$	$0,5 \times 10 = 5$	4	$5 > 4$: défaut
C	aucun (B est encore actif)	0	3	tient

$\Rightarrow D_1 = \{A, B\}$: 1 nouveau défaut.

A-B-C pas à pas

Données : A doit 10 à B ; B doit 8 à C ; $c_A = 5$, $c_B = 4$, $c_C = 3$; $\lambda = 0,5$; origine $s = A$.

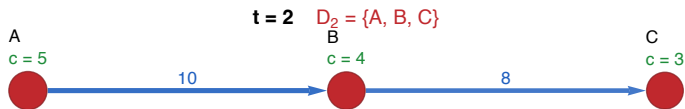


Étape 2 : on part de $D_1 = \{A, B\}$; seule C est encore active.

banque j	débiteurs de j dans D_1	perte $L_j(D_1)$	capital c_j	verdict
C	B, qui lui doit $e_{BC} = 8$	$0,5 \times 8 = 4$	3	$4 > 3$: défaut

$\Rightarrow D_2 = \{A, B, C\}$: 1 nouveau défaut.

Données : A doit 10 à B ; B doit 8 à C ; $c_A = 5$, $c_B = 4$, $c_C = 3$; $\lambda = 0,5$; origine $s = A$.



Étape 3 : $D_2 = \{A, B, C\}$; plus aucune banque active, rien ne peut changer : $D_3 = D_2$, c'est le **point fixe**.

Résultat

$D_{\text{final}} = \{A, B, C\}$; nouveaux_par_etape = [1, 1] ; $y = \frac{3-1}{3-1} = 1$: la cascade atteint toutes les banques. C'est le test 1 de `verifier_simulateur.py`.

Récapitulatif ($\lambda = 0,5$, origine A)

t	D_t	pertes des actives	nouveaux
0	{A}	$L_B = 5 > 4$, $L_C = 0$	B
1	{A, B}	$L_C = 4 > 3$	C
2	{A, B, C}	(aucune active)	arrêt

Ligne t : pertes calculées à partir de D_t ; elles donnent D_{t+1} .

Variantes (même réseau)

- $\lambda = 0,3$: B perd $3 \leq 4$; aucun défaut secondaire, $y = 0$.
- $\lambda = 0,4$: B perd exactement $4 = c_B$: pas de défaut, $y = 0$. Dès que $\lambda > 0,4$, toute la chaîne tombe ($y = 1$) : un **effet de seuil**.
- Origine B ($\lambda = 0,5$) : C perd $4 > 3$; A ne perd rien (elle est débitrice de B) : $y = 1/2$.
- Origine C : C ne doit rien à personne : $y = 0$.

À retenir

La même règle donne des cascades très différentes selon λ et selon l'origine : c'est ce que le modèle de la partie 4 devra apprendre à anticiper.

Démonstration en direct : notebook wolfram/Demos_Contagion.nb, **Démo 2 : A-B-C**

Ce qu'on va observer

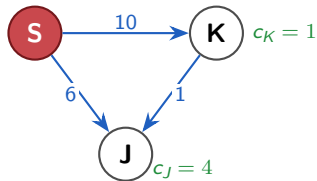
- À chaque étape, la perte $L_j(D_t)$ de chaque banque active, comparée à son capital c_j .
- Le passage au rouge d'une banque quand sa perte dépasse son capital.
- L'effet d'un changement de λ ou de banque d'origine.

Questions à se poser

- À partir de quel λ la banque B tombe-t-elle ? ($10\lambda > 4$, soit $\lambda > 0,4$.)
- Que se passe-t-il exactement à $\lambda = 0,4$?
- Si c_C valait 5, jusqu'où irait la cascade à $\lambda = 0,5$?
- Pourquoi l'origine C ne provoque-t-elle rien ?



Piège 1 : cumuler les pertes d'une étape à l'autre



S doit 6 à J et 10 à K ; K doit 1 à J ; $\lambda = 0,5$; origine S (test 7).

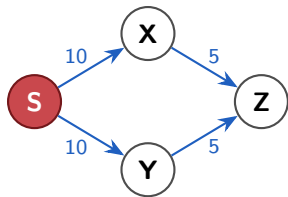
étape	J : perte totale (juste)	J : cumul (faux)
1 ($D_0 = \{S\}$)	$0,5 \times 6 = 3 \leq 4$	3
2 ($D_1 = \{S, K\}$)	$0,5 \times (6 + 1) = 3,5 \leq 4$	$3 + 3,5 = 6,5 > 4$

À l'étape 1, K perd $0,5 \times 10 = 5 > 1$ et tombe, ce qui déclenche une étape 2. Juste : J tient toujours, résultat $\{S, K\}$ et [1]. Faux : J « tombe » à l'étape 2.

Règle

$L_j(D_t)$ est la perte **totale** due à *toutes* les banques de D_t : recalculez-la à chaque étape à partir de D_t . En l'ajoutant à la perte de l'étape précédente, la dette de S envers J serait comptée deux fois.

Piège 2 : ajouter les défauts au fil de l'eau



$cx = cy = cz = 4$; $\lambda = 0,5$; origine S
(test 4).

Juste (ajouts simultanés)

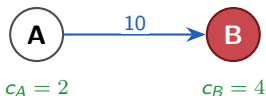
- Étape 1, à partir de $D_0 = \{S\}$: X et Y perdent $5 > 4$ et tombent ; Z perd 0.
- Étape 2, à partir de D_1 : Z perd $0,5 \times (5 + 5) = 5 > 4$ et tombe.
- Résultat : $[2, 1]$.

Faux (mise à jour séquentielle) : on ajoute X à D dès qu'il tombe, puis Y, pendant le même parcours ; Z « voit » déjà X et Y et tombe à l'étape 1 : $[3]$.

Règle

Calculez tous les défauts d'une étape dans un ensemble à part, nouveaux, à partir de D_t seulement ; ajoutez-les ensuite en une fois. Sinon, nouveaux_par_etape dépend de l'ordre de parcours des banques. (L'ensemble final, lui, reste le même ici.)

Piège 3 : inverser le sens des arêtes



A doit 10 à B ; $\lambda = 0,5$ (test 6).

- Origine B : B ne doit rien à A, donc **A ne perd rien** : résultat $\{B\}$ et $[]$.
- Origine A : B perd $0,5 \times 10 = 5 > 4$: résultat $\{A, B\}$ et $[1]$.
- Avec le sens inversé, l'origine B ferait perdre 5 à A (> 2) : faux défaut.

Règle

Pour la perte de j , parcourez ses **débiteurs** : $G.predecessors(j)$ (les i tels que $i \rightarrow j$), avec $G[i][j][\text{"montant"}]$. $G.successors(j)$ (ses créanciers) inverserait la règle.

Deux oublis fréquents (test 8)

L'origine tombe même avec un capital de 10^9 ; les banques isolées restent dans le graphe et dans N .

Piège 4 : les arrondis des nombres flottants

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.3 * 10.30          # exact : 3,09
3.0900000000000003
>>> 0.3 * 10.30 > 3.09    # faux défaut !
True
>>> from aides_contagion import depasse_capital
>>> depasse_capital(0.3 * 10.30, 3.09)
False
>>> 0.3 * 10 == 3.0       # tombe juste
True
```

- L'ordinateur code les décimaux en binaire (**nombres flottants**) : 0,1, 0,3 ou 10,30 n'y ont pas d'écriture exacte. Les calculs sont très précis, mais pas exacts.
- Or le défaut est **strict** : une perte égale au capital ne déclenche rien. Une erreur de 10^{-16} suffit à inverser le verdict.
- `depasse_capital(perte, capital)` teste $\text{perte} > \text{capital} + 10^{-9} \times \max(1, |\text{capital}|)$: une marge relative minuscule qui absorbe les arrondis sans changer les vrais verdicts.

Test 3

$0,5 \times 10 = 5$ et $0,3 \times 10 = 3$ contre un capital égal : pas de défaut ; puis trois vrais pièges ($0,3 \times 10,30$, $0,6 \times 5,15$, $0,9 \times 2,10$ contre 3,09, 3,09, 1,89). Toujours `depasse_capital`, jamais `>` ni `==`.

La mesure principale : la proportion de défauts secondaires

Intuition : on mesure ce que le réseau *ajoute* au choc initial, pas le choc lui-même.

$$y = \frac{|D_{\text{final}}| - 1}{N - 1}$$

où D_{final} : ensemble final des défauts (le point fixe), origine comprise

$|\cdot|$: nombre d'éléments d'un ensemble

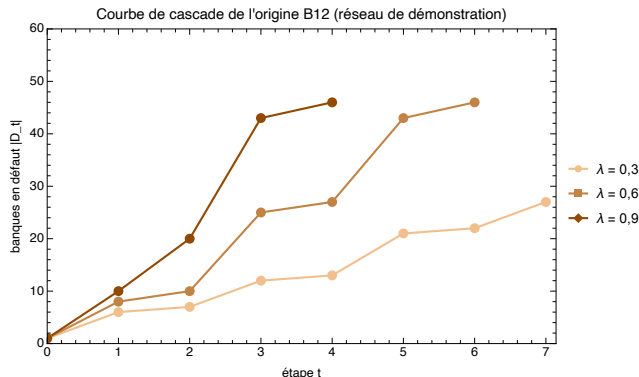
-1 : on retire l'origine, dont le défaut est imposé

N : nombre de banques du réseau, banques isolées comprises ($N = 60$ ici)

y : part des $N - 1$ autres banques qui font défaut, entre 0 et 1

- $y = 0$: le choc reste limité à l'origine.
- $y = 1$: toutes les autres banques tombent.
- Aide : `proportion_secondaire(defauts, n)`.
- A-B-C : $(3 - 1)/(3 - 1) = 1$.
- 60 banques, 15 défauts secondaires : $15/59 \approx 0,254$.
- RELAIS, relais renforcé : $4/11 \approx 0,364$.

La courbe de cascade



Réseau de démonstration hors base : une même origine, trois valeurs de λ .

- La **courbe de cascade** d'un scénario trace le nombre de banques en défaut $|D_t|$ en fonction de l'étape t , de $|D_0| = 1$ jusqu'au point fixe.
- Elle se déduit de nouveaux_par_etape :
`np.cumsum([1] + nouveaux_par_etape) ; RELAIS`
donne 1, 5, 6, 12.
- Plus λ est grand, plus la cascade est rapide et étendue ; les paliers et les sauts traduisent des groupes de banques qui tombent ensemble.
- Une courbe de y en fonction de λ complète utilement la lecture.

Démonstration en direct : notebook wolfram/Demos_Contagion.nb, **Démo 3 : cascade sur 60 banques**, curseurs λ et origine

Ce qu'on va observer

- La propagation étape par étape sur le réseau de démonstration (hors base).
- La courbe de cascade qui change quand on déplace le curseur λ .
- Des origines voisines qui produisent des cascades très différentes.

Questions à se poser

- Pour une origine donnée, existe-t-il un λ à partir duquel la cascade « décolle » ?
- Les banques qui ont le plus de créanciers produisent-elles toujours les plus grandes cascades ?
- Combien d'étapes dure une grande cascade ?
- Quelles banques ne tombent jamais, et pourquoi ($\lambda w_{\text{in}}(j) \leq c_j$) ?

Ce qu'il faut enregistrer pour chaque scénario

Pour chaque scénario (réseau, origine s , λ) : l'origine, λ , l'ensemble final des défauts et le nombre de nouveaux défauts à chaque étape; y s'en déduit.

Contrat : `simuler(G, origine, lam) -> (defauts, nouveaux_par_etape)`

`defauts` : ensemble (set) des banques en défaut, origine comprise. `nouveaux_par_etape` : la liste des $|D_t \setminus D_{t-1}|$ pour $t = 1, \dots, T$, sans l'origine et sans zéro final. A-B-C : (`{ "A", "B", "C" }`, `[1, 1]`); aucune propagation : (`{s}`, `[]`).

```
from aides_contagion import generer_scenarios
scenarios = generer_scenarios(banques, expositions, construire_graphe, simuler,
                             repartition=repartition)      # 12 x 60 x 3 = 2 160 lignes
scenarios.to_csv("scenarios.csv", index=False)             # relire avec keep_default_na=False
```

reseau_id	usage	origine	lambda	n_defauts	y	n_etapes	nouveaux_par_etape	defauts_finaux
ABC		A	0.6	3	1.0	2	1;1	A;B;C
RELAIS		N1	0.6	12	1.0	3	4;1;6	N1;N2;...;S6

$n_defauts = |D_{final}|$ (origine comprise); n_etapes = nombre d'étapes avec au moins un nouveau défaut.

Vérifier le simulateur : les 9 tests (1/2)

Test	Ce qui est vérifié	Erreur détectée
1 Exemple A-B-C	$\lambda = 0,5$, origine A : {A, B, C} et [1, 1]	règle, sens, arrêt
2 Aucun défaut secondaire	$\lambda = 0$; réseau sans arête ; capitaux supérieurs à toute perte possible	pertes « fantômes »
3 Perte égale au capital	$0,5 \times 10 = 5 = c$ et $0,3 \times 10 = 3 = c$: pas de défaut ; trois pièges flottants ($0,3 \times 10,30$ contre 3,09, etc.)	$\geq$ au lieu de $>$, arrondis
4 Défauts simultanés	S doit 10 à X et à Y, qui doivent chacun 5 à Z ($c = 4$, $\lambda = 0,5$) : [2, 1]	mise à jour séquentielle
5 Monotonie	sur des réseaux aléatoires, augmenter des capitaux n'agrandit jamais l'ensemble final	capital lu ailleurs que dans G, cumul

Lancer les tests. Dans un notebook : `sys.path.append("aides")`, puis `from verifier_simulateur import verifier` et `verifier(simuler, construire_graphe)`, qui affiche un rapport et renvoie True si les 9 tests passent. En ligne de commande : `python aides/verifier_simulateur.py mon_module`.

Vérifier le simulateur : les 9 tests (2/2)

Test	Ce qui est vérifié	Erreur détectée
6 Sens des arêtes	A doit 10 à B ; origine B : A ne perd rien ; origine A : B tombe	arêtes inversées
7 Pas de double comptage	une perte de premier tour inférieure au capital n'est pas recomptée au tour suivant	pertes cumulées
8 Origine imposée, banques isolées	l'origine tombe même avec un capital de 10^9 ; une banque isolée ne tombe jamais ; construire_graphe la conserve	oubli de l'origine, graphe sans les isolées
9 Irréversibilité et arrêt	point fixe atteint, origine incluse, $\sum$ nouveaux_par_etape = $ D - 1$, au plus $N - 1$ étapes, chaque défaut justifié	arrêt trop tôt, boucle infinie, comptage faux

Méthode

Écrivez simuler, retrouvez A-B-C à la main puis avec le code, lancez les 9 tests, corrigez un test à la fois : pour chaque échec, le rapport indique ce qui était attendu, ce que votre fonction a renvoyé et l'erreur la plus probable.

Entrées : graphe $G(c_j, e_{ij})$, origine s , fraction de perte λ

Sorties : ensemble final D , liste nouveaux_par_etape

```
1  $D \leftarrow \{s\}$  ▷  $D_0$  : défaut imposé
2 nouveaux_par_etape  $\leftarrow []$ 
3 répéter
4   nouveaux  $\leftarrow \emptyset$ 
5   pour chaque banque  $j \notin D$  faire
6      $L_j \leftarrow \lambda \times \sum \{e_{ij} : i \in D, i \rightarrow j\}$  ▷ recalculée
7     si  $L_j > c_j$  alors ajouter  $j$  à nouveaux
8   fin pour
9   si nouveaux =  $\emptyset$  alors sortir ▷ point fixe
10   $D \leftarrow D \cup \text{nouveaux}$  ▷ ajout simultané
11  ajouter  $|\text{nouveaux}|$  à nouveaux_par_etape
12 fin répéter
13 renvoyer  $D$ , nouveaux_par_etape
```

- Ligne 1 : choc imposé (test 8).
- Ligne 5 : seules les banques actives sont testées (irréversibilité).
- Ligne 6 : perte totale à partir de D entier, jamais cumulée (test 7); débiteurs de j seulement (test 6).
- Ligne 7 : comparaison **stricte**, avec depasse_capital (test 3).
- Lignes 4 et 10 : nouveaux défauts mis de côté, puis ajoutés ensemble (test 4).
- Ligne 9 : arrêt au point fixe (test 9).

Python	Rôle
<code>G.nodes</code>	toutes les banques du réseau, isolées comprises
<code>G.predecessors(j)</code>	les débiteurs i de j (arêtes $i \rightarrow j$)
<code>G[i][j]["montant"]</code> , <code>G.nodes[j]["capital"]</code>	e_{ij} et c_j
<code>defaults = {origine} ; j in defaults</code>	un ensemble Python (set); test d'appartenance
<code>nouveaux = set() ; nouveaux.add(j)</code>	ensemble vide; ajout d'un élément
<code>defaults = nouveaux ; len(nouveaux)</code>	réunion en place; nombre d'éléments
<code>sum(... for i in ... if ...)</code>	somme sur une partie des débiteurs
<code>depasse_capital(perte, capital)</code>	la comparaison stricte, robuste aux arrondis
<code>while True: ... break</code>	répéter jusqu'au point fixe

Piste du sujet

Utilisez deux ensembles, les défauts connus et les nouveaux défauts; parcourez les débiteurs de chaque banque encore active, puis faites l'union. Ne modifiez jamais `G` dans `simuler` : les aides le vérifient.

Ce que le modèle ignore

- Les **paiements partiels** : une banque en difficulté rembourse souvent une partie de ses dettes (Eisenberg et Noe) ; ici, c'est tout ou rien.
- Les ventes d'actifs, la baisse des prix, la liquidité, la panique : aucune réaction des marchés.
- Une fraction λ identique et connue, des capitaux et un réseau figés, un seul choc initial.

Ce qu'il permet

- Isoler le rôle de la structure du réseau dans la contagion directe.
- Produire des effets de seuil et des cascades très inégales selon l'origine et λ .
- Comparer des stratégies avec des règles du jeu identiques.

À retenir (partie 3)

Perte totale $L_j(D_t) = \lambda \sum_{i \in D_t} e_{ij}$, recalculée à chaque étape ; défaut si $L_j(D_t) > c_j$ (strictement) ; ajouts simultanés ; défaut irréversible ; point fixe en au plus $N - 1$ étapes ; $y = (|D_{\text{final}}| - 1) / (N - 1)$.
Un mécanisme pédagogique, pas une prévision de crise réelle.

- 0 Ouverture
- 1 Contexte historique et économique
- 2 Graphes orientés et expositions
- 3 Pertes, capital, cascade à seuil
- 4 Machine learning**
- 5 Interventions et exemple RELAIS
- 6 Organisation

Pourquoi un modèle de prédiction ?

Le simulateur donne y pour n'importe quel scénario. La partie 4 pose une autre question : peut-on **prévoir** y *avant* de lancer la cascade, à partir d'informations disponibles sur le réseau initial ?

- Le modèle approche le résultat du simulateur ; surtout, il aide à comprendre quelles caractéristiques d'une banque d'origine annoncent une grande cascade.
- Il n'est demandé ni de prévoir une crise réelle, ni de montrer un gain de vitesse.

Apprentissage supervisé

On dispose d'exemples dont on connaît la réponse. On apprend une règle f qui relie des **variables explicatives** x (les entrées) à une **cible** y (la réponse), puis on l'applique à des cas nouveaux : la **prédiction** est $\hat{y} = f(x)$. Ici, y est un nombre entre 0 et 1 : c'est un problème de **régression**.

Comme en économétrie, on pense $y = f(x) + \varepsilon$; mais f est ici flexible (arbre, forêt) et l'on juge le modèle sur ses prédictions pour des cas *jamaïs vus*, pas sur des coefficients ni sur la causalité.

Une ligne par triplet (réseau, banque d'origine, λ) : $\lambda \in \{0,3; 0,6; 0,9\}$ et chaque banque sert d'origine, soit $12 \times 60 \times 3 = 2160$ scénarios sans intervention. Le simulateur fournit la cible y ; `generer_scenarios` organise les calculs.

suivi (jamais en entrée)			variables explicatives $\times$ (réseau initial)					cible
reseau_id	usage	origine	lambda	capital_origine	degre_sortant	poids_sortant	...	y
R..	apprentissage	B01	0.3	c_{B01}	$d_{out}(B01)$	$w_{out}(B01)$	...	y
R..	apprentissage	B01	0.6	c_{B01}	$d_{out}(B01)$	$w_{out}(B01)$	...	y
R..	apprentissage	B01	0.9	c_{B01}	$d_{out}(B01)$	$w_{out}(B01)$	...	y
R..	apprentissage	B02	0.3	c_{B02}	$d_{out}(B02)$	$w_{out}(B02)$	...	y
⋮								⋮

- Chaque origine apparaît trois fois : ses variables de réseau sont identiques, seule λ change.
- Les 180 lignes d'un réseau partagent la même structure : elles ne sont pas indépendantes (d'où le partage par réseau, plus loin).

Colonne	Définition	Intuition économique
lambda	λ du scénario	intensité du choc : chaque défaut coûte plus cher
capital_origine	c_s	ne protège pas s (défaut imposé), mais peut refléter sa taille ou sa position
degre_entrant	$d_{in}(s)$, nombre de débiteurs de s	position de s : prêteur de nombreuses banques ?
degre_sortant	$d_{out}(s)$, nombre de créanciers de s	banques touchées dès la première étape
poids_entrant	$w_{in}(s)$, créances détenues par s	taille de s comme prêteur
poids_sortant	$w_{out}(s)$, dettes de s	pertes infligées à la première étape : $\lambda w_{out}(s)$ au total

- Toutes se calculent sur le **réseau initial**, avant tout choc : elles sont disponibles au moment de prédire.
- s est la banque d'origine. Son capital peut être corrélé à d'autres caractéristiques, mais ne la protège pas du défaut initial imposé.

$$\text{atteignables_2}(s) = |\{j \neq s : \text{un chemin de 1 ou 2 arêtes va de } s \text{ à } j\}|$$
$$\text{fragilite_moy_creanciers}(s) = \frac{1}{d_{\text{out}}(s)} \sum_{j: s \rightarrow j} v_j \quad \text{avec} \quad v_j = \frac{w_{\text{in}}(j)}{c_j}$$

où `atteignables_2` : créanciers de s et créanciers de ses créanciers (s exclue) : portée possible de la cascade en deux étapes
 v_j : fragilité de j (partie 5), moyennée sur les créanciers directs de s , 0 s'il n'y en a pas : des créanciers fragiles relaient la cascade

Contrôles : A-B-C, origine A : 2 atteignables (B, C), fragilité moyenne $v_B = 2,5$; RELAIS, origine N1 : 11 atteignables, fragilité moyenne $(4 \times 8 + 2)/5 = 6,8$.

En Python : quelques lignes avec NetworkX (banques à distance 1 ou 2 de s , successeurs de s) ; le calcul est à écrire par votre équipe et à vérifier sur les deux contrôles ci-dessus.

Ce qui est interdit en entrée du modèle

- **Les identifiants** (`reseau_id`, `origine`) servent au suivi, jamais à la prédiction : « B07 » d'un réseau n'a rien à voir avec « B07 » d'un autre ; un modèle qui s'en sert apprend par cœur.
- **Tout résultat de la cascade** : `n_defaults`, `n_etapes`, `nouveaux_par_etape`, `defaults_finaux`, `y...`, et toute variable qui *recalcule* un morceau de la cascade.
- Piège subtil : « nombre de créanciers j de s tels que $\lambda e_{sj} > c_j$ » semble calculable sur le réseau initial... mais c'est exactement le **nombre de défauts à la première étape**, que le sujet interdit.

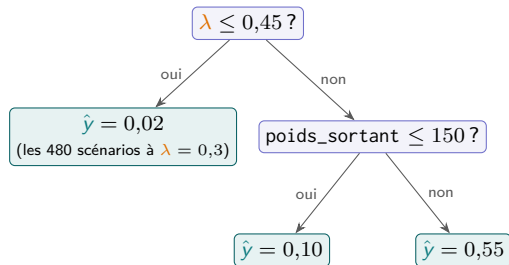
Fuite de données

Une **fuite de données** est une information utilisée par le modèle qui ne serait pas disponible au moment de prédire, ou qui provient de la cible ou des données de test. Elle rend les performances trompeusement bonnes.

Question à se poser pour chaque variable

Pourrais-je la calculer pour une banque d'un réseau *nouveau*, sans lancer le simulateur ? Si non, elle est interdite.

L'arbre de régression : une suite de questions



Arbre de profondeur 2 ; valeurs illustratives. En apprentissage, chaque valeur de λ compte $8 \times 60 = 480$ scénarios.

- Intuition : une suite de questions oui/non sur les variables ; chaque question coupe les scénarios en deux groupes plus homogènes.
- Une **feuille** (case terminale) prédit la **moyenne** des y d'apprentissage qui y tombent.
- Lecture : un scénario à $\lambda = 0,6$ dont l'origine doit plus de 150 reçoit $\hat{y} = 0,55$.
- **Profondeur** : nombre maximal de questions successives ; profondeur d : au plus 2^d feuilles. Plus l'arbre est profond, plus il est flexible.

Comment l'arbre choisit ses questions

À chaque nœud, l'algorithme essaie chaque variable x_k et chaque seuil τ , et garde le découpage qui réduit le plus l'**erreur quadratique** :

$$\min_{k, \tau} \sum_{r \in G} (y_r - \bar{y}_G)^2 + \sum_{r \in D} (y_r - \bar{y}_D)^2 \quad G = \{r : x_{rk} \leq \tau\}, \quad D = \{r : x_{rk} > \tau\}$$

où r, y_r : un scénario d'apprentissage et sa cible simulée
 x_{rk}, τ : valeur de la variable k pour le scénario r ; seuil candidat
 G, D : scénarios envoyés à gauche ($\leq \tau$) et à droite ($> \tau$)
 $\bar{y}_G, \bar{y}_D$: moyennes de y dans chaque groupe : les prédictions des deux nouvelles feuilles

- On recommence dans chaque groupe jusqu'à la profondeur maximale (max_depth) ou un effectif minimal par feuille (min_samples_leaf). Méthode **gloutonne** : le meilleur découpage immédiat, sans retour en arrière.
- Atouts : aucune normalisation des variables; capte les effets de seuil et les interactions ($\lambda \times$ structure). Limites : prédiction en escalier, instable (un petit changement des données peut changer l'arbre).

- Idée : moyenner beaucoup d'arbres *différents* pour réduire l'instabilité d'un arbre seul.
- Chaque arbre apprend sur un **échantillon bootstrap** : autant de lignes que l'apprentissage, tirées au hasard *avec remise* (certaines plusieurs fois, d'autres jamais).
- À chaque découpage, l'arbre n'examine qu'un sous-ensemble aléatoire des variables (`max_features`) : les arbres se ressemblent moins.

$$\hat{y} = \frac{1}{M} \sum_{m=1}^M f_m(x)$$

où $f_m(x)$: prédiction du m -ième arbre pour les variables x d'un scénario
 M : nombre d'arbres (`n_estimators`, par exemple 300); $\hat{y}$: leur moyenne

- Atouts : plus stable, souvent plus précise. Limites : moins lisible, plus lente; une graine (`random_state=0`) rend les résultats reproductibles.
- `RandomForestRegressor(n_estimators=300, max_depth=6, random_state=0)` : même usage qu'un arbre.

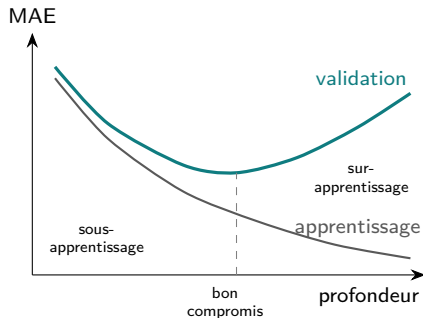


Schéma de principe (pas des données).

- **Surapprentissage** : le modèle colle aux particularités, et même aux hasards, des données d'apprentissage au lieu d'apprendre des régularités. Il prédit très bien ce qu'il a vu, mal ce qu'il n'a pas vu.
- **Signe** : erreur d'apprentissage très faible, erreur de validation nettement plus forte.
- **Remèdes** : limiter la complexité (`max_depth`, `min_samples_leaf`), moyenner (forêt), et toujours juger sur des données non vues.
- Ici, le risque est accru : 2 160 lignes, mais seulement 12 réseaux, et les lignes d'un même réseau se ressemblent.

Une **référence** est une prédiction très simple, à battre : sans elle, une MAE ne veut rien dire.

- **Globale** : pour tout scénario, la médiane des y d'apprentissage.
- **Par λ** : la médiane des y d'apprentissage qui ont le même λ (trois valeurs).
- La seconde tient compte de l'intensité du choc. Si votre modèle ne la bat pas, les variables du réseau n'apportent rien au-delà de λ .

Aides : `refs = references_medians(app)` donne
`{"globale": ..., "par_lambda": {0.3: ..., 0.6: ..., 0.9: ...}}`; `predire_references(refs, val)`
renvoie les colonnes `pred_globale` et `pred_par_lambda`.

Pourquoi la médiane ?

Parmi les prédictions *constantes*, la **médiane** (valeur qui partage les données en deux moitiés) minimise l'erreur absolue moyenne ; la moyenne, elle, minimise l'erreur quadratique.

Intuition : relever la prédiction d'un cran réduit l'erreur des points situés au-dessus et augmente d'autant celle des points situés en dessous ; on y gagne tant qu'il y a plus de points au-dessus. L'optimum en laisse autant de chaque côté.

Exemple : $y = 0; 0; 0; 0,1; 1$. Médiane 0 : $MAE = 0,22$. Moyenne 0,22 : $MAE = 0,312$.

Le protocole : apprentissage, validation, test, par réseau

Apprentissage (8 réseaux) : on ajuste le modèle (les découpages de l'arbre).

Validation (2 réseaux) : on compare quelques réglages, les **hyperparamètres** (par exemple la profondeur), et on choisit.

Test (2 réseaux) : on mesure, une seule fois à la fin, l'erreur sur des réseaux jamais vus.



partage par réseau : correct



partage par lignes : trompeur

■ apprentissage
■ validation
■ test
colonne = un réseau ;
case = des scénarios

Répartition fixée dans `repartition.csv` (colonne usage) : respectez-la ; tous les scénarios d'un réseau restent ensemble. Avec un partage des *lignes*, le modèle a déjà « vu » chaque réseau de test (mêmes banques, mêmes liens) : l'erreur mesurée est trop optimiste et ne dit rien de sa capacité à traiter un **nouveau réseau**.

Régler sur la validation, tester une seule fois

- 1 Choisir les variables, avec des arguments économiques (et l'apprentissage seulement).
- 2 Apprendre deux ou trois réglages sur l'apprentissage, par exemple $\text{max_depth} \in \{3, 5, 8\}$.
- 3 Calculer la MAE de chaque réglage et des deux références sur la validation ; retenir le meilleur réglage (à égalité, le plus simple).
- 4 **Figurer** tous les choix : variables, modèle, réglage.
- 5 Évaluer **une seule fois** sur le test : MAE globale et par réseau, comparée aux deux références.

Règles du sujet

Un seul modèle appris suffit, sans recherche exhaustive. Les transformations éventuelles (normalisation...) sont ajustées sur l'apprentissage seul ; aucun réglage ne dépend du test.

Pourquoi

Si l'on modifie le modèle après avoir regardé le test, le test devient une seconde validation : l'erreur annoncée est optimiste.

$$\text{MAE} = \frac{1}{m} \sum_{r=1}^m |\hat{y}_r - y_r|$$

où

- m : nombre de scénarios évalués ; sur le test : $2 \times 180 = 360$
- $\hat{y}_r$: prédiction du modèle pour le scénario r
- y_r : valeur simulée pour le scénario r
- $|\cdot|$: valeur absolue : une erreur de $+0,1$ et une de $-0,1$ comptent toutes deux $0,1$

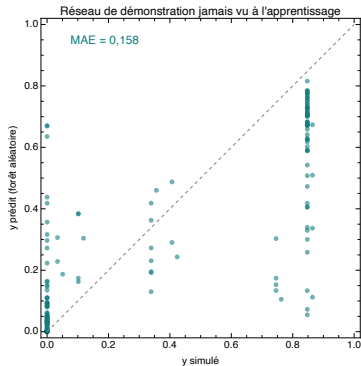
- **MAE** (*mean absolute error*, erreur absolue moyenne) : même unité que y . Une MAE de $0,05$ est une erreur moyenne de **5 points de pourcentage** sur la proportion de défauts secondaires.

- À présenter : MAE globale sur le test, MAE par réseau de test (180 scénarios chacun), et les mêmes pour les deux références.

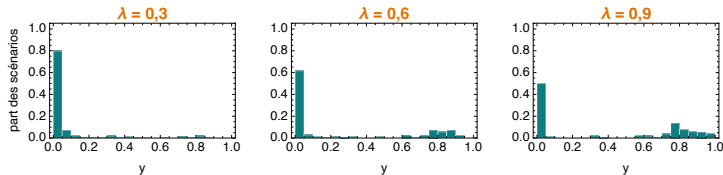
	test	réseau 1	réseau 2
référence globale	...	...	...
référence par λ	...	...	...
votre modèle	...	...	...

Deux réseaux de test issus d'un générateur : une évaluation limitée. Un gain n'est pas garanti et n'est pas une condition de réussite.

Analyser les erreurs



Prédit contre simulé : réseau de démonstration jamais vu à l'apprentissage (hors base).



Histogrammes de y pour chaque λ (réseaux de démonstration hors base).

- Un point par scénario ; la diagonale $\hat{y} = y$ est la prédiction parfaite ; les points éloignés sont les grosses erreurs.
- La distribution de y peut être très déséquilibrée (beaucoup de 0, un groupe de grandes cascades) : regardez-la avant d'interpréter une MAE.
- Commentez au moins **deux grosses erreurs** : quelle origine, quel λ ? que valent ses variables ? que montre nouveaux_par_etape ? quelle information manquait au modèle (un effet de seuil, un relais à trois arêtes...) ?

Avant d'annoncer une MAE, vérifiez qu'aucune de ces fuites n'est présente :

- ☐ une variable calculée à partir de la cascade (défauts de la première étape, `n_defaults...`) ou de `y` ;
- ☐ un identifiant (`reseau_id`, `origine`) parmi les variables explicatives ;
- ☐ un partage aléatoire des lignes au lieu du partage par réseau de `repartition.csv` ;
- ☐ des références, normalisations ou sélections de variables calculées sur toutes les données plutôt que sur l'apprentissage seul ;
- ☐ un réglage choisi, ou modifié, après avoir regardé le test ;
- ☐ un test utilisé plusieurs fois pour comparer des variantes.

Symptôme

Une MAE de test spectaculairement basse. Réflexe : relire la liste des colonnes d'entrée et le filtrage par usage.

Voir aussi la documentation de scikit-learn : « Common pitfalls and recommended practices » (fuites de données).

Python : construire la table des variables

```
lignes = []
for r, banques_r in banques.groupby("reseau_id"):
    G = construire_graphe(banques_r, expositions[expositions["reseau_id"] == r])
    for s in G.nodes:
        lignes.append({"reseau_id": r, "origine": s,
                        "capital_origine": G.nodes[s]["capital"],
                        "degre_entrant": G.in_degree(s),
                        "degre_sortant": G.out_degree(s),
                        "poids_entrant": G.in_degree(s, weight="montant"),
                        "poids_sortant": G.out_degree(s, weight="montant")})
variables_banques = pd.DataFrame(lignes)

# 2 160 lignes : chaque scénario reçoit les variables de son origine (réseau initial)
table = scenarios.merge(variables_banques, on=["reseau_id", "origine"], how="left")
VARIABLES = ["lambda", "capital_origine", "degre_entrant", "degre_sortant",
             "poids_entrant", "poids_sortant"]  # + vos variables motivées
```

groupby parcourt les réseaux; merge donne à chaque scénario les variables de son origine. Vérifiez ensuite : 2 160 lignes, aucune valeur manquante.

Python : références, réglage et évaluation

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_absolute_error
from aides_contagion import references_medianes, predire_references

app, val, test = (table[table["usage"] == u] for u in ("apprentissage", "validation", "test"))

refs = references_medianes(app)                # médianes de l'apprentissage seulement
pv = predire_references(refs, val)             # colonnes pred_globale et pred_par_lambda
print(mean_absolute_error(val["y"], pv["pred_globale"]),
      mean_absolute_error(val["y"], pv["pred_par_lambda"]))

for p in (3, 5, 8):                            # deux ou trois profondeurs, jugées sur la validation
    arbre = DecisionTreeRegressor(max_depth=p, random_state=0).fit(app[VARIABLES], app["y"])
    print(p, mean_absolute_error(val["y"], arbre.predict(val[VARIABLES])))

# Choix figés (ici la profondeur 5, par exemple) : UNE évaluation sur le test, globale et par réseau
arbre = DecisionTreeRegressor(max_depth=5, random_state=0).fit(app[VARIABLES], app["y"])
erreur = (arbre.predict(test[VARIABLES]) - test["y"]).abs()
print(erreur.mean(), erreur.groupby(test["reseau_id"]).mean())
```

Références sur le test : `predire_references(refs, test)`. Forêt : `RandomForestRegressor`.

- 0 Ouverture
- 1 Contexte historique et économique
- 2 Graphes orientés et expositions
- 3 Pertes, capital, cascade à seuil
- 4 Machine learning
- 5 Interventions et exemple RELAIS**
- 6 Organisation

Décider avant le choc, avec un budget limité

La cellule de supervision peut renforcer le capital de quelques banques **avant** de connaître l'origine du choc. Les dettes et la règle de propagation ne changent pas.

$$B = 0,10 \sum_j c_j \quad \text{réparti entre } k = 5 \text{ banques : chacune reçoit } B/k$$

où B : budget du réseau : 10 % de la somme des capitaux
 c_j : capitaux **initiaux**, avant tout renforcement
 k : nombre de banques renforcées (distinctes); B/k est le renfort de chacune

- Aide : `capitaux_renforces(G, choisies, part_budget=0.10)` renvoie une copie renforcée de G ; passez toujours le graphe **initial**, sinon B serait calculé sur des capitaux déjà renforcés.
- Le défaut initial reste imposé, même si l'origine a reçu du capital : le renforcement agit sur la contagion *secondaire*.
- RELAIS : $\sum_j c_j = 46,5$, donc $B = 4,65$; dans cet exemple, $k = 1$ (tout le budget à une banque).

- ❶ **Sans intervention** : capitaux initiaux.
- ❷ **Aléatoire** : 5 banques distinctes tirées au sort, sans remise. On utilise les 10 tirages officiels (graines 0 à 9) de `tirages_aleatoires.csv`, communs à toutes les équipes.
- ❸ **Ciblée** : 5 banques choisies par une règle que vous justifiez à partir du réseau *avant* le choc.

Règles d'équité (sujet)

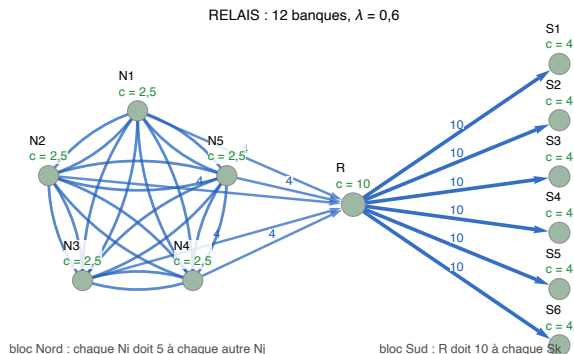
- Règle définie sur les réseaux d'apprentissage et de validation, puis **figée** avant l'évaluation finale sur les réseaux de test.
- Pour un réseau : banques choisies **une seule fois**, avant les chocs, puis évaluées sur les mêmes 60 origines $\times$ 3 valeurs de λ , soit 180 scénarios.
- Aléatoire : chaque tirage est évalué sur tous les scénarios du réseau ; on présente la moyenne et la dispersion des 10 tirages.
- Même budget pour toutes les stratégies ; défaut initial toujours imposé.

$$v_j = \frac{w_{\text{in}}(j)}{c_j} = \frac{\sum_i e_{ij}}{c_j} \quad s_j = v_j \cdot d_{\text{out}}(j)$$

où v_j : **fragilité** de j : ses créances rapportées à son capital (combien de fois il pourrait perdre son capital)
 $d_{\text{out}}(j)$: nombre de créanciers de j : banques touchées directement si j tombe
 s_j : **score de ciblage** : on renforce les k banques de plus grand score

- Idée : protéger une banque susceptible de **subir** des pertes (fragile) et de les **transmettre** (nombreux créanciers). Une banque fragile sans créancier ($d_{\text{out}}(j) = 0$) a un score nul : son défaut ne se propage pas.
- Rappel : si $\lambda v_j \leq 1$, la banque j ne peut pas tomber à ce λ .
- Ce score **n'est pas optimal** (voir RELAIS). Vous pouvez choisir un autre critère motivé : précisez le classement et le départage des égalités (par exemple banque_id croissant).

L'exemple RELAIS : le réseau



12 banques (fichiers
exemples/relais_*.csv) :

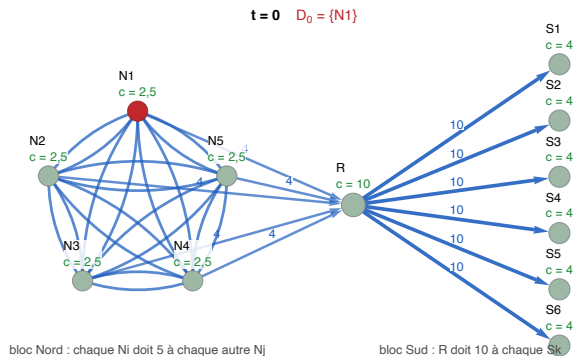
- **Bloc Nord** $N_1 \dots N_5$: chaque N_i doit 5 à chacune des quatre autres ;
 $c_N = 2,5$.
- **Relais R** : chaque N_i doit 4 à R ;
 $c_R = 10$.
- **Bloc Sud** $S_1 \dots S_6$: R doit 10 à chaque S_k ; $c_S = 4$.

$\sum_j c_j = 5 \times 2,5 + 10 + 6 \times 4 = 46,5$, donc
 $B = 4,65$, attribué en entier à **une** banque
($k = 1$).

Scénario étudié : origine N_1 , $\lambda = 0,6$.

Quelle banque renforcer ?

RELAIS sans intervention (origine N1, $\lambda = 0,6$)

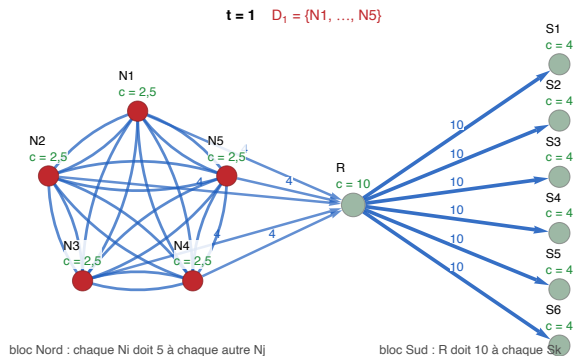


Départ : $D_0 = \{N1\}$, choc imposé.

Capitaux : $c_N = 2,5$, $c_R = 10$, $c_S = 4$.

On applique la règle, étape par étape, sans aucun renforcement.

RELAIS sans intervention (origine N1, $\lambda = 0,6$)

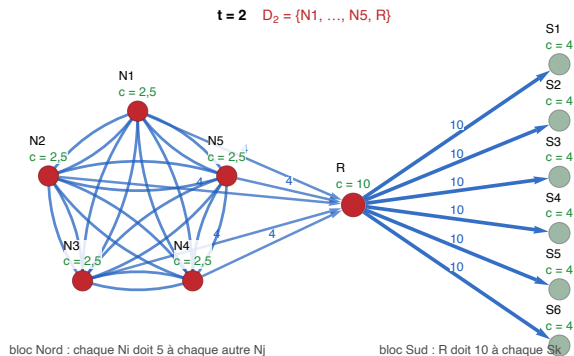


Étape 1 (à partir de D_0) :

- chaque N_j ($j = 2, \dots, 5$) perd $0,6 \times 5 = 3 > 2,5$: **défaut** ;
- R perd $0,6 \times 4 = 2,4 \leq 10$: tient ;
- les S_k ne perdent rien (R est actif).

4 nouveaux défauts.

RELAIS sans intervention (origine N1, $\lambda = 0,6$)

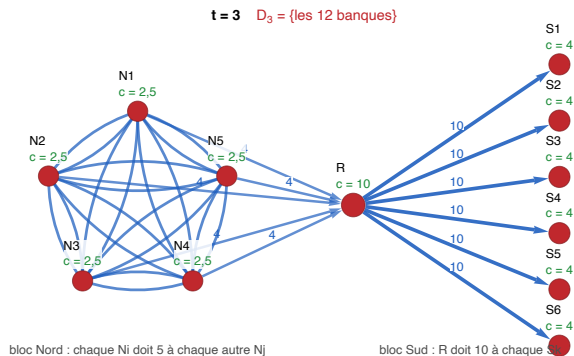


Étape 2 (à partir de D_1 , tout le Nord) :

- R perd $0,6 \times (5 \times 4) = 12 > 10$:
défaut.

1 nouveau défaut.

RELAIS sans intervention (origine N1, $\lambda = 0,6$)



Étape 3 (à partir de D_2) :

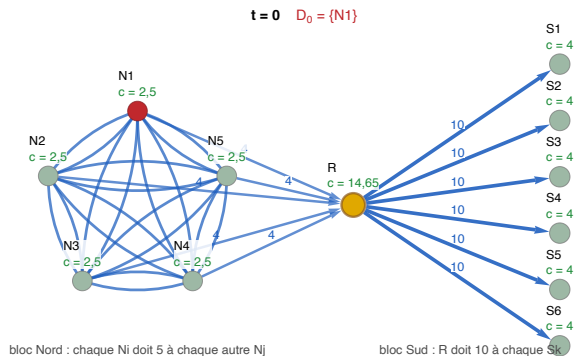
- chaque S_k perd $0,6 \times 10 = 6 > 4$: **défaut.**

6 nouveaux défauts, puis point fixe.

Résultat

$[4, 1, 6]$; $y = 11/11 = 1$: tout le réseau tombe.

RELAIS avec le relais renforcé (origine N1, $\lambda = 0,6$)



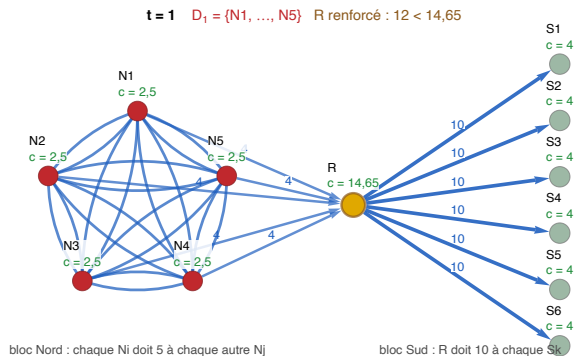
Départ : même choc, $D_0 = \{N1\}$.

Tout le budget va à R :

$$c_R = 10 + 4,65 = 14,65.$$

En Python : `H = capitaux_renforces(G, ["R"])`, puis `simuler(H, "N1", 0.6)`.

RELAIS avec le relais renforcé (origine N1, $\lambda = 0,6$)



Étape 1 : comme avant, N2...N5 tombent (4 défauts).

Étape 2 : R perd $0,6 \times 20 = 12 \leq 14,65$: **il tient**. Le Sud ne perd rien : point fixe.

Résultat

[4]; $y = 4/11 \approx 0,36$ au lieu de 1 : renforcer le relais **coupe** la cascade.

Trois régimes selon λ

λ	perte de chaque N_j (étape 1)	perte de R (Nord tombé)	sans intervention	R renforcé ($c_R = 14,65$)
0,3	$1,5 \leq 2,5$: tient	(le Nord tient)	$[], y = 0$	$y = 0$: intervention inutile
0,6	$3 > 2,5$: défaut	$12 > 10$, mais $12 \leq 14,65$	$[4, 1, 6], y = 1$	$[4], y = 4/11$: cascade arrêtée
0,9	$4,5 > 2,5$: défaut	$18 > 14,65$	$[4, 1, 6], y = 1$	$[4, 1, 6], y = 1$: budget insuffisant

Question : quel renfort minimal ?

À $\lambda = 0,6$, il faut $c_R \geq 12$ (une perte égale au capital ne fait pas tomber) : un renfort de 2 suffit. À $\lambda = 0,9$, il faudrait 8, plus que $B = 4,65$.

La même intervention peut être inutile, décisive ou insuffisante selon l'intensité du choc : présentez toujours les résultats **par** λ .

Le score naïf ne choisit pas le relais

	$w_{\text{in}}(j)$	c_j	v_j	$d_{\text{out}}(j)$	s_j
N_i (Nord)	$4 \times 5 = 20$	2,5	8	5	40
R (relais)	$5 \times 4 = 20$	10	2	6	12
S_k (Sud)	10	4	2,5	0	0

Le score place les cinq banques Nord en tête (égalité à 40, départagée par banque_id : N1), puis R, puis le Sud.

Avec $k = 1$, il renforce donc N1 ($c_{N1} = 7,15$) :

- choc sur N1 : rien ne change, son défaut est imposé ;
- choc sur une autre banque Nord : N1 résiste à l'étape 1 (perte 3) mais tombe à l'étape 2 (perte 12) ; la cascade n'est que retardée : [3, 1, 1, 6], $y = 1$.

R est un **verrou** : le seul passage entre le Nord et le Sud.

Leçon

Le score repère des banques fragiles et connectées, pas les points de passage obligés : il n'est pas optimal. Toute règle se vérifie avec le simulateur.

Prédire un risque n'est pas mesurer l'effet d'une intervention

- Dans RELAIS à $\lambda = 0,6$, les banques Nord sont les origines les plus contagieuses ($y = 1$) ; R l'est moins ($y = 6/11 \approx 0,55$). Pourtant c'est R qu'il faut renforcer, et renforcer N1 ne change rien.
- Le modèle de la partie 4 prédit y pour une origine donnée, *dans le réseau tel qu'il est* : il capte une **association** entre caractéristiques et cascades.
- Une intervention *modifie* le réseau (les capitaux). Son effet est une question **causale** : « que se passerait-il si l'on renforçait ces banques ? ». On y répond en re-simulant avec les mêmes règles : c'est un **contrefactuel**.

Conséquences pour votre étude

- L'efficacité d'une stratégie se vérifie **toujours** avec le simulateur, jamais avec le modèle de prédiction.
- Une banque très contagieuse n'est pas forcément celle qu'il faut renforcer en priorité : discutez cette différence.
- Une stratégie ciblée peut être peu efficace ; une conclusion négative correctement établie est un résultat recevable.

Pour chaque stratégie : la moyenne de y sur les scénarios, la plus grande cascade observée, les résultats par λ , et le gain moyen par rapport à l'absence d'intervention, en **points de pourcentage** : $100 \times (\text{moyenne sans} - \text{moyenne de la stratégie})$. Pour l'aléatoire : moyenne et dispersion des 10 tirages.

RELAIS ($k = 1$; 12 origines $\times 3 \lambda$)	y moyen	y max	$\lambda = 0,3$	$\lambda = 0,6$	$\lambda = 0,9$	gain (points)
sans intervention	0,308	1	0	0,462	0,462	réf.
N1 renforcée (choix du score)	0,308	1	0	0,462	0,462	0,0
R renforcé	0,220	1	0	0,197	0,462	8,8

- Le gain moyen (8,8 points) cache des régimes très différents : 26,5 points à $\lambda = 0,6$, rien à 0,3 ni à 0,9.
- La moyenne sur les scénarios est un indicateur de comparaison, pas une probabilité de crise réelle.

```
from aides_contagion import charger_tirages, comparer_interventions, resumer_interventions

tirages = charger_tirages("donnees/tirages_aleatoires.csv") # {(reseau_id, graine): [5 banques]}
strategies = {
    "sans": None, # capitaux initiaux
    "aleatoire": [tirages[(r, g)] for g in range(10)], # les 10 tirages officiels du réseau r
    "ciblee": ma_regle(G, k=5), # VOTRE règle, figée avant le test
}
resultats = comparer_interventions(G, simuler, strategies) # G initial ; 60 origines x 3 lambda
resultats.insert(0, "reseau_id", r) # utile pour empiler plusieurs réseaux
resume = resumer_interventions(resultats)
resume[["strategie", "y_moyen", "y_max", "y_moyen_ecart_type", "gain_pp"]]
```

- comparer_interventions applique le budget (capitaux_renforces) à chaque sélection et simule tous les scénarios; le résumé donne aussi y_moyen_lambda_0.3, etc.
- ma_regle est à écrire : elle choisit les banques une fois pour toutes, à partir du réseau initial, sans connaître l'origine du choc.

Démonstration en direct : notebook wolfram/Demos_Contagion.nb, **Démo 4 : RELAIS**

Ce qu'on va observer

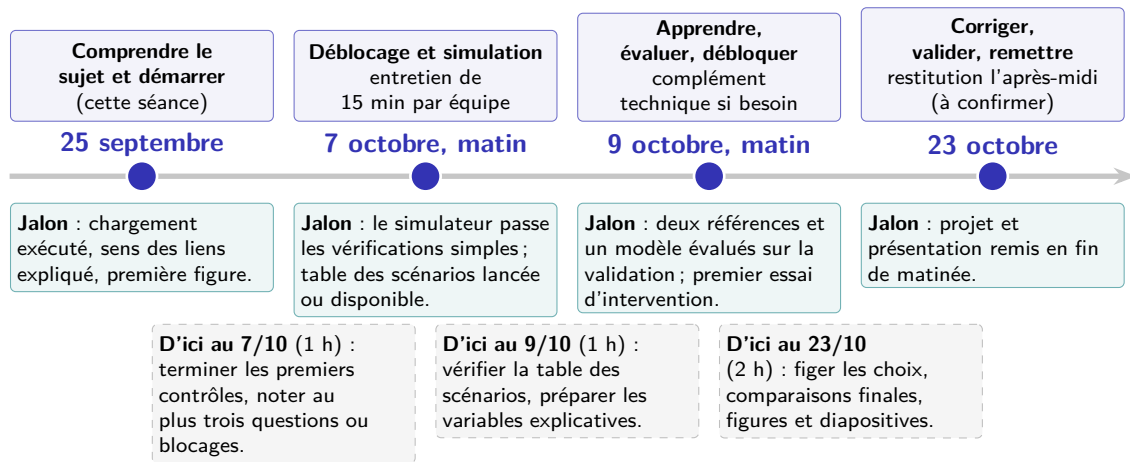
- La cascade sans intervention : le Nord, puis R, puis le Sud.
- La même cascade avec R renforcé : elle s'arrête au Nord.
- Le curseur λ : 0,3, puis 0,6, puis 0,9.
- Le renforcement d'une banque Nord à la place de R.

Questions à se poser

- Pourquoi R, et pas une banque Nord ?
- Quel renfort minimal de R faut-il à $\lambda = 0,6$? à $\lambda = 0,9$?
- Quelle information sur le réseau permettrait de repérer un verrou comme R ?
- Que devient la conclusion si le choc part de R ?

- 0 Ouverture
- 1 Contexte historique et économique
- 2 Graphes orientés et expositions
- 3 Pertes, capital, cascade à seuil
- 4 Machine learning
- 5 Interventions et exemple RELAIS
- 6 Organisation**

Les jalons du hackathon



Charge visée : environ 12 heures en séances accompagnées et 4 heures de travail autonome par étudiant.
Restitution : 8 minutes de présentation et 4 minutes d'échange par équipe ; chaque membre doit pouvoir expliquer une partie de la démarche.

Un dossier ou un dépôt partagé, **entièrement réexécutable**, contenant :

- ❶ **Le travail Python** : un notebook structuré, exécutable du début à la fin, avec au besoin quelques fonctions dans des fichiers .py (un projet en scripts convient aussi).
- ❷ **Un court fichier de présentation** : auteurs, instructions d'exécution, dépendances, graines aléatoires, hypothèses, aides ou ressources externes utilisées.
- ❸ **Les résultats** : une visualisation de réseau, une courbe de cascade, la distribution des cibles, la comparaison des prédictions aux valeurs simulées, un tableau ou un graphique comparant les interventions.
- ❹ **Cinq à six diapositives** : question étudiée, méthode, principaux résultats, recommandation et limites. Aucun rapport supplémentaire.

Style attendu et marge de liberté

Noms explicites, quelques fonctions réutilisables, les vérifications de la partie 3 ; ni architecture complexe, ni déploiement, ni tableau de bord. Chaque membre comprend les choix principaux. Vous choisissez visualisations, variables, modèle et règle de ciblage ; une fois le socle terminé, **un seul** prolongement : faire varier le budget, un second modèle de régression, ou une autre structure de réseau.

Critère	Poids	Ce qui est évalué
Données et réseau	15 %	contrôles, orientation des liens, indicateurs interprétés
Simulation	25 %	règle correcte, cas simples vérifiés, résultats cohérents
Machine learning	25 %	cible et variables pertinentes, référence, partage par réseau, évaluation et analyse des erreurs
Intervention et analyse	20 %	budget identique, comparaison équitable, conclusion argumentée et limites explicites
Reproductibilité et présentation	15 %	exécution complète, figures lisibles, présentation claire et compréhension collective

Ce qui compte

La qualité du raisonnement compte davantage que le nombre de modèles ou d'options. Une stratégie ciblée peut être peu efficace : une conclusion négative correctement établie est un résultat recevable. Les prolongements facultatifs ne donnent pas de points supplémentaires.

Demandé par le sujet

- ☐ Chargement des données exécuté (jalon du jour).
- ☐ Sens des liens expliqué : $i \rightarrow j$, i doit à j .
- ☐ Une première figure.
- ☐ Premiers contrôles terminés (et essai sur brutes/).
- ☐ Au plus trois questions ou blocages notés.

Conseillé pour l'entretien du 7 octobre

- ☐ A-B-C déroulé à la main, puis simuler commencé.
- ☐ `verifier_simulateur.py` lancé, même si tout ne passe pas.
- ☐ À montrer : un résultat, un blocage reproductible (code, message d'erreur), votre prochaine étape.

Pour démarrer dès maintenant

Télécharger le dépôt GitHub (diapositive 7), lire README.md, charger les données, lancer les contrôles, construire le graphe d'un réseau d'apprentissage, tracer une première figure. Si le simulateur bloque le 7 octobre, une table de secours est remise, mais il reste à le corriger.

Apprentissage (ensemble d') les 8 réseaux sur lesquels le modèle est ajusté.

Apprentissage supervisé apprendre, sur des exemples dont la réponse est connue, une règle qui relie des variables explicatives à une cible.

Arbre de régression modèle qui prédit par une suite de questions oui/non ; chaque feuille prédit une moyenne.

Arête orientée lien d'un sommet vers un autre ; ici $i \rightarrow j$: i doit à j .

Banque isolée banque sans dette ni créance ; elle compte dans N .

Bootstrap échantillon de même taille tiré au hasard avec remise.

Budget B 10 % de la somme des capitaux initiaux d'un réseau, réparti entre $k = 5$ banques.

Capital c_j fonds propres : montant de pertes que j peut absorber sans faire défaut.

Cascade de défauts enchaînement de défauts, chacun provoqué par les pertes que les précédents infligent.

Cascade à seuil modèle où une banque fait défaut dès que sa perte dépasse un seuil, son capital.

Chemin suite d'arêtes qui s'enchaînent ($i \rightarrow j \rightarrow k$).

Cible y grandeur à prédire : la proportion de défauts secondaires.

Cœur-périphérie structure où un petit groupe très interconnecté (le cœur) est entouré de banques reliées surtout au cœur.

Contagion transmission d'une difficulté d'une banque à d'autres.

Contrefactuel ce qui se passerait si l'on changeait une chose (par exemple un capital), le reste étant égal ; on le mesure en re-simulant.

Contrepartie l'autre partie d'un contrat financier.

Créance somme qu'une banque doit recevoir d'une autre.

Créancier celui à qui l'on doit (j dans $i \rightarrow j$).

Débiteur celui qui doit (i dans $i \rightarrow j$).

Défaut incapacité à honorer ses engagements ; ici, perte > capital, ou choc initial.

Degré entrant $d_{\text{in}}(j)$ nombre d'arêtes qui arrivent en j : ses débiteurs.

Degré sortant $d_{\text{out}}(j)$ nombre d'arêtes qui partent de j : ses créanciers.

Dette somme qu'une banque doit à une autre.

Ensemble collection d'éléments sans ordre ni répétition ; en Python, set.

Exposition e_{ij} montant que j peut perdre si i fait défaut, c'est-à-dire ce que i doit à j .

Feuille case terminale d'un arbre ; elle donne la prédiction.

Forêt aléatoire moyenne de nombreux arbres, appris sur des échantillons bootstrap et des sous-ensembles de variables.

Fraction de perte λ part d'une créance perdue quand le débiteur fait défaut ; $1 - \lambda$ est le taux de recouvrement.

Fragilité v_j créances détenues rapportées au capital : $w_{in}(j)/c_j$.

Fuite de données information utilisée par le modèle qui ne serait pas disponible au moment de prédire, ou qui vient de la cible ou du test.

Graine aléatoire entier qui fixe une suite de tirages « au hasard » pour la rendre reproductible.

Graphe orienté pondéré sommets reliés par des arêtes orientées qui portent un poids.

Hyperparamètre réglage fixé avant l'apprentissage (par exemple la profondeur) ; il se choisit sur la validation.

Irréversibilité une banque en défaut le reste.

MAE erreur absolue moyenne, $\frac{1}{m} \sum_r |\hat{y}_r - y_r|$; $0,05 = 5$ points de pourcentage.

Matrice d'exposition E tableau $N \times N$: lignes débiteurs, colonnes créanciers, cases e_{ij} .

Médiane valeur qui partage un ensemble de nombres en deux moitiés.

Nombre flottant représentation binaire approchée d'un nombre décimal dans l'ordinateur.

Point fixe état qui ne change plus quand on applique la règle : $D_{t+1} = D_t$.

Poids entrant $w_{\text{in}}(j)$ somme des montants des arêtes entrantes : créances de j .

Poids sortant $w_{\text{out}}(j)$ somme des montants des arêtes sortantes : dettes de j .

Profondeur nombre maximal de questions successives d'un arbre.

Référence prédiction simple à battre : médiane globale, médiane par λ .

Régression apprentissage supervisé dont la cible est un nombre.

Scénario triplet (réseau, banque d'origine s , λ).

Score s_j $v_j \cdot d_{\text{out}}(j)$: piste de ciblage suggérée, non optimale.

Sommet élément d'un graphe ; ici une banque.

Surapprentissage le modèle colle aux données d'apprentissage et généralise mal.

Test (ensemble de) les 2 réseaux utilisés une seule fois, pour l'évaluation finale.

Validation (ensemble de) les 2 réseaux qui servent à choisir les réglages.

Variable explicative information d'entrée du modèle, notée x .

Ventes forcées ventes d'actifs en urgence qui font baisser les prix (canal non modélisé).

Cascade à seuil

$$L_j(D_t) = \lambda \sum_{i \in D_t} e_{ij}$$

$$D_{t+1} = D_t \cup \{j \notin D_t : L_j(D_t) > c_j\}$$

$D_0 = \{s\}$; arrêt quand $D_{t+1} = D_t$ (au plus $N - 1$ étapes).

Mesure principale

$$y = \frac{|D_{\text{final}}| - 1}{N - 1}$$

$N = 60$ banques, isolées comprises ; $y \in [0, 1]$.

Réseau

$d_{\text{in}}(j)$: nombre de débiteurs ; $d_{\text{out}}(j)$: nombre de créanciers

$w_{\text{in}}(j) = \sum_i e_{ij}$: créances de j ; $w_{\text{out}}(j) = \sum_k e_{jk}$: dettes de j

Prédiction

$$\text{MAE} = \frac{1}{m} \sum_{r=1}^m |\hat{y}_r - y_r|$$

Intervention

$B = 0,10 \sum_j c_j$; B/k par banque

$v_j = w_{\text{in}}(j)/c_j$ $s_j = v_j d_{\text{out}}(j)$

Conventions

- $i \rightarrow j$ de poids e_{ij} : i doit e_{ij} à j ; la perte suit la flèche.
- Défaut strict ($>$), ajouts simultanés, irréversible; origine imposée; N compte les isolées; $\lambda \in \{0,3; 0,6; 0,9\}$.
- Protocole : 8 / 2 / 2 réseaux, deux références médianes, test utilisé une fois. Interventions : $k = 5$, même budget, 10 tirages officiels, 180 scénarios par réseau.

Contrat Python

- `construire_graphe(banques_r, expositions_r) -> nx.DiGraph; attributs "capital" (sommets) et "montant" (arêtes).`
- `simuler(G, origine, lam) -> (defaults, nouveaux_par_etape); A-B-C : ({ "A", "B", "C" }, [1, 1]); sans propagation : ({s}, []).`
- `depasse_capital(perte, capital)` pour tester un défaut.

Code couleur

λ	orange	E07000
e_{ij}	bleu	1F5FBF
c_j	vert	1E8C3A
D_t	rouge	C0282D
L_j	violet	7B3FA0
$y, \hat{y}$	sarcelle	0F7C80
v_j, s_j, B	brun	8C5A1E

- Acemoglu D., Ozdaglar A., Tahbaz-Salehi A. (2015), « Systemic Risk and Stability in Financial Networks », *American Economic Review*, 105(2), 564-608.
- Allen F., Gale D. (2000), « Financial Contagion », *Journal of Political Economy*, 108(1), 1-33.
- Battiston S., Puliga M., Kaushik R., Tasca P., Caldarelli G. (2012), « DebtRank : Too Central to Fail ? Financial Networks, the FED and Systemic Risk », *Scientific Reports*, 2, 541.
- Craig B., von Peter G. (2014), « Interbank Tiering and Money Center Banks », *Journal of Financial Intermediation*, 23(3), 322-347.
- Eisenberg L., Noe T. H. (2001), « Systemic Risk in Financial Systems », *Management Science*, 47(2), 236-249.
- Furfine C. H. (2003), « Interbank Exposures : Quantifying the Risk of Contagion », *Journal of Money, Credit and Banking*, 35(1), 111-128.
- Gai P., Kapadia S. (2010), « Contagion in Financial Networks », *Proceedings of the Royal Society A*, 466(2120), 2401-2423.
- Haldane A. G. (2009), « Rethinking the Financial Network », discours, Financial Student Association, Amsterdam, avril 2009.
- Upper C., Worms A. (2004), « Estimating Bilateral Exposures in the German Interbank Market : Is There a Danger of Contagion ? », *European Economic Review*, 48(4), 827-849.

Pour le contexte historique : President's Working Group on Financial Markets (1999), *Hedge Funds, Leverage, and the Lessons of Long-Term Capital Management*; Financial Crisis Inquiry Commission (2011), *The Financial Crisis Inquiry Report*.

Pour le code : documentations de NetworkX (graphes, sommets, arêtes et attributs) et de scikit-learn (arbres de décision, mesures d'évaluation et références, fuites de données).